Blackboard Exercise 2: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Blackboard Exercise 2

From Fundamentals to AI Apocalypse · personal study guide

Contents

Exercise 2.1: Natural numbers 2

Exercise 2.2: Floating-point numbers 3

Exercise 2.3: Signed and unsigned integers 3

Exercise 2.4: Text representation and interpretation 4

Exercise 2.5: Boolean algebra 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

A worked route through number bases, binary fractions, integer encodings, ASCII and Boolean algebra. Try each subpart before opening its steps.

Exercise 2.1: Natural numbers

Connected topic: Number systems and data representation (§3.1–3.3)

2.1.1 Number representations

Try first: Complete the five conversions; show how the first and last binary rows convert directly to octal and hexadecimal.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.2 Range of an unsigned representation

Try first: Find the ranges for 8, 16, and 32 bits, derive the general rule, then find the fewest bits that cover 0 through 1000.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.3 Representation and storage

Try first: Decide which listed values fit an unsigned byte, identify equal values, and explain why distinct spellings can denote the same number.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.4 Arithmetic

Try first: Calculate the four sums, difference, and products in the operand bases without converting the operands to decimal first; decide whether each result fits an unsigned byte.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.5 Overflow

Try first: For each decimal addition, find the mathematical sum, whether an 8-bit unsigned overflow occurs, and the stored byte. Then explain the NumPy result for uint8(250)+uint8(13).

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.6 Working with an unknown radix

Try first: The numeral 243 in base r has decimal value 99. Find r and decide whether 354_r and 358_r are valid.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

Made by Thanate Amornjirapong (Focus)

Exercise 2.2: Floating-point numbers

Connected topic: Binary fractions and floating-point representation (§4.1)

2.2.1 Binary fractions

Try first: Complete the four binary/decimal conversions and show one calculation in each direction.

Answers: see "Exercise 2.2: Floating-point numbers" under Answers at the end.

2.2.2 Finite and repeating representations

Try first: Convert 0.1_{10} until its repeating pattern is visible. Decide which listed fractions terminate in binary, then state the general condition.

Answers: see "Exercise 2.2: Floating-point numbers" under Answers at the end.

Exercise 2.3: Signed and unsigned integers

Connected topic: Integer representations and arithmetic (§3.4)

2.3.1 Two's complement

Try first: Complete the table for the three bit patterns and the two decimal values, using exactly eight bits.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

2.3.2 Range and interpretation

Try first: Find the 8-bit and general n-bit ranges, explain the asymmetric counts, and interpret 11110110_2 both ways.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

2.3.3 Signed overflow

Try first: Add 01100101_2 and 00111110_2, interpret all three patterns as signed decimal, and test for signed overflow.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

Made by Thanate Angornjiraporn (Focus)

Exercise 2.4: Text representation and interpretation

Connected topic: Text and character representation (§3.5)

2.4.1 ASCII

Try first: Complete the table and encode AIBE as four 8-bit ASCII bytes.

Answers: see "Exercise 2.4: Text representation and interpretation" under Answers at the end.

2.4.2 One bit pattern, different meanings

Try first: Interpret 01000001_2 as unsigned, as 8-bit two's complement, and as ASCII. State what extra information a reader needs.

Answers: see "Exercise 2.4: Text representation and interpretation" under Answers at the end.

Exercise 2.5: Boolean algebra

Connected topic: Boolean algebra identities (§1.1–1.2)

2.5.1 Truth table

Try first: Fill the columns A', $A{\cdot}B$, $A{+}B$, and $A{+}(B{\cdot}C)$ for all input rows in the given order.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

2.5.2 Simplification

Try first: Simplify the four expressions and name the Boolean law used at each step.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

2.5.3 Equivalence and construction

Try first: Compare $A{+}(B{\cdot}C)$ with $(A{+}B)(A{+}C)$ using algebra and a truth table; construct $F{=}1$ exactly when at least two inputs are 1.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

Made by Thanate Angkrnjiraporn (Focus)

Answers

Exercise 2.1: Natural numbers

2.1.1 Number representations · worked solution

1. Read each subscript as the base. For binary-to-octal, group bits from the binary point in threes; for binary-to-hexadecimal, group in fours. Pad only on the left with zeroes when needed.
2. $10110110_2 = 010\ 110\ 110_2 = 266_8$, and $1011\ 0110_2 = B6_{16}$. The complete table is: $10110110_2 = 266_8 = 182_{10} = B6_{16}$; $347_8 = 11100111_2 = 231_{10} = E7_{16}$; $156_{10} = 10011100_2 = 234_8 = 9C_{16}$; $2D7_{16} = 0010\ 1101\ 0111_2 = 1327_8 = 727_{10}$; $110101011001_2 = 6531_8 = D59_{16} = 3417_{10}$.
3. The groups work because $8 = 2^3$ and $16 = 2^4$: each octal digit encodes exactly three bits, and each hexadecimal digit exactly four. Check by expanding any one row back into powers of its base.

Exercise 2.1: Natural numbers

2.1.2 Range of an unsigned representation · worked solution

1. An n-bit unsigned pattern has place values 2^0 through 2^{n-1}. Its minimum is all zeroes; its maximum is their sum.
2. The ranges are 0–255, 0–65,535, and 0–4,294,967,295. In general: $0 \leq x \leq 2^n-1$. For 0 through 1000, $2^9-1=511$ is too small, while $2^{10}-1=1023$ is enough, so 10 bits are required.
3. The count of distinct patterns is 2^n. Verify minimality by showing $2^9 < 1001 \leq 2^{10}$.

Exercise 2.1: Natural numbers

2.1.3 Representation and storage · worked solution

1. An unsigned byte stores 0 through 255. Convert each representation to its value, then compare with that range.
2. 11111111_2=255, 10000000_2=128, FF_{16}=255, and 256_{10}=256. The first and third denote the same value and both fit; 128 fits; 256 does not.
3. A numeral's value is the weighted sum of its digits in its base. Base-2 11111111 and base-16 FF are different spellings but both sum to 255. A byte stores the value's bit pattern, not the textual spelling.

Exercise 2.1: Natural numbers

2.1.4 Arithmetic · worked solution

1. Keep the radix fixed during each operation, align place values, and perform carries or borrows in that radix.
2. $10110110_2 + 00101101_2 = 11100011_2$ (227), which fits. $11001010_2 - 00110111_2 = 10010011_2$ (147), which fits. $00101101_2 \times 00000100_2 = 10110100_2$ (180), which fits. $A7_{16} + 3C_{16} = E3_{16}$ (227), which fits.
3. All four values are at most FF_{16}=255. The subtraction is nonnegative; the multiplication by 4 is a two-bit left shift, which provides a quick check.

Exercise 2.1: Natural numbers

2.1.5 Overflow · worked solution

1. An 8-bit unsigned value stores the sum only when it is at most 255. For modular storage, retain the low eight bits, equivalently take the remainder modulo 256.
2. 200+55=255: no overflow, stored 255. 200+56=256: overflow, stored 0. 250+13=263: overflow, stored 7. 128+128=256: overflow, stored 0. The condition is a+b>255. For the NumPy expression, the expected 8-bit result is 7 because 263 mod 256=7; this is fixed-width unsigned arithmetic; the NumPy 2.5.4 run also emits a RuntimeWarning for scalar overflow.
3. Compare each stored value with its mathematical sum modulo 256. The boundary case 200+55 is exactly 255, so it does not overflow.

Exercise 2.1: Natural numbers

2.1.6 Working with an unknown radix · worked solution

1. Expand by place value: $243_r = 2r^2+4r+3$. A radix must be an integer larger than every digit.
2. Solve $2r^2+4r+3=99$, giving $r^2+2r-48=0=(r+8)(r-6)$. The positive radix is r=6. In base six, 354_6 is valid because each digit is below 6; 358_6 is invalid because digit 8 is not allowed.
3. Substitute r=6: $2\cdot36+4\cdot6+3=99$. Reject -8 because a radix is positive.

Exercise 2.2: Floating-point numbers

2.2.1 Binary fractions · worked solution

1. After the binary point, places have weights 2^{-1}, 2^{-2}, and so on. To convert a binary fraction, add the place values of its 1 bits.
2. 0.1011_2=1/2+1/8+1/16=0.6875_{10}. 10.011_2=2+1/4+1/8=2.375_{10}. For the reverse direction, repeatedly double the fractional part: 0.625×2=1.25 gives bit 1 and remainder .25; .25×2=.5 gives 0; .5×2=1 gives 1. Thus 0.625_{10}=0.101_2. Similarly 5.375_{10}=101.011_2.
3. Convert back: 0.101_2=1/2+1/8=0.625, and 101.011_2=4+1+1/4+1/8=5.375.

Exercise 2.2: Floating-point numbers

2.2.2 Finite and repeating representations · worked solution

1. Repeatedly multiply the fractional remainder by 2; each product's integer part is the next bit.
2. For 1/10: remainders produce $0.0001100110011\ldots_2 = 0.0(0011)_2$, with the block 0011 repeating. Reduce the candidates: 1/2, 3/8, 1/10, 5/16, 7/20. The finite ones are 1/2, 3/8, and 5/16; 1/10 and 7/20 repeat.
3. A reduced fraction p/q has a finite binary expansion exactly when q is a power of 2. Here $10=2\cdot5$ and $20=4\cdot5$ retain a factor 5 after reduction, so those expansions cannot terminate.

Exercise 2.3: Signed and unsigned integers

2.3.1 Two's complement · worked solution

1. For a nonnegative value, write its ordinary binary form padded to eight bits. For a negative value, invert the bits of its magnitude and add one.
2. $01111111_2=127$; $11111111_2=-1$; $10000000_2=-128$. Conversely, -8 is 11111000_2, and -37 is 11011011_2.
3. Check negative encodings by interpreting the most significant bit as weight -128 and the remaining bits as positive powers: $11111000_2=-128+120=-8$; $11011011_2=-128+64+16+8+2+1=-37$.

Exercise 2.3: Signed and unsigned integers

2.3.2 Range and interpretation · worked solution

1. In n-bit two's complement, the leading bit has weight -2^{n-1}; the other bits sum from 0 to $2^{n-1}-1$.
2. The 8-bit range is -128 through 127. Generally it is -2^{n-1} through $2^{n-1}-1$. There are 2^{n-1} negative patterns but only $2^{n-1}-1$ positive patterns because zero uses one of the nonnegative patterns. 11110110_2 is 246 unsigned and -10 signed $(-128+64+32+16+4+2)$.
3. The same stored bits have no inherent signedness. The surrounding type or format decides whether the leading bit has weight $+128$ or -128.

Exercise 2.3: Signed and unsigned integers

2.3.3 Signed overflow · worked solution

1. Add the bit patterns: $01100101_2+00111110_2=10100011_2$.
2. The operands are $+101$ and $+62$; the 8-bit result pattern 10100011_2 represents -93 $(-128+32+2+1)$. The mathematical sum is 163, outside the range $-128\ldots127$, so signed overflow occurs.
3. Two positive operands produced a negative result, which detects overflow. In general, signed addition overflows when operand signs match and the result sign differs; opposite-sign operands cannot overflow when added.

Exercise 2.4: Text representation and interpretation

2.4.1 ASCII · worked solution

1. Treat each ASCII code as a byte. Convert binary to decimal and look up the corresponding standard ASCII character; reverse the process for a given decimal code.
2. 01000001_2=65_{10}='A'; 01100001_2=97_{10}='a'; 48_{10}=00110000_2='0'. AIBE is 01000001 01001001 01000010 01000101_2, or 41 49 42 45 in hexadecimal.
3. Check that uppercase and lowercase letters differ by 32 in ASCII (one bit at this position), and that decimal 48 is the character '0', not the numeric value zero.

Exercise 2.4: Text representation and interpretation

2.4.2 One bit pattern, different meanings · worked solution

1. Evaluate the same eight bits under each encoding rule rather than changing the bits.
2. Unsigned interpretation: 65. Two's-complement interpretation: 65, because the leading bit is zero. ASCII interpretation: 'A'.
3. The data's type or agreed encoding supplies the meaning: for example, unsigned byte, signed 8-bit integer, or ASCII character. Bits alone do not specify their interpretation.

Exercise 2.5: Boolean algebra

2.5.1 Truth table · worked solution

1. Evaluate each row independently with 0=false and 1=true. Complement flips a bit; · is AND; + is OR. Parentheses make B·C happen before the outer OR.
2. In the row order 000,001,010,011,100,101,110,111, the columns are: A' = 1,1,1,1,0,0,0,0; A·B = 0,0,0,0,0,0,1,1; A+B = 0,0,1,1,1,1,1,1; A+(B·C) = 0,0,0,1,1,1,1,1.
3. Spot-check 011: A=0 and B·C=1, so the final expression is 1. For all rows with A=1, both OR columns are 1.

Exercise 2.5: Boolean algebra

2.5.2 Simplification · worked solution

1. Use complement, identity, distributive, and absorption laws; show the intermediate rewrite so each result can be checked.
2. A+A'=1 by complement. A·A'=0 by complement. A+(A'·B)=(A+A')(A+B)=1·(A+B)=A+B, by distributivity and identity. (A+B)(A+B')=A+BB'=A+0=A, by distributivity and complement.
3. Check the last two with A=0 and A=1 separately; their outputs match A+B and A respectively.

Exercise 2.5: Boolean algebra

2.5.3 Equivalence and construction · worked solution

1. Expand the product using distributivity, then use idempotence and absorption. For the construction, list the input rows with at least two 1s.
2. $(A+B)(A+C)=A\cdot A+A\cdot C+A\cdot B+B\cdot C=A+A\cdot C+A\cdot B+B\cdot C=A+B\cdot C$. Thus the expressions are equivalent. For row order 000 through 111, both evaluate to 0,0,0,1,1,1,1,1.
3. The true rows for the "at least two" function are 011, 101, 110, and 111, giving $F=A'BC+AB'C+ABC'+ABC$. Grouping pairs or using absorption yields $F=AB+AC+BC$. Check: exactly two 1s makes one pair term true; three 1s makes all three true.

Made by Thanate Amornjiraporn (Focus)

Personal worked solutions based on the APDR Blackboard Exercise 2 sheet, written in my own words; no exercise-sheet prose or figures are reproduced. Not an official FAU publication; check course details against the current materials. Made by **Thanate Amornjiraporn (Focus)**.